

Vidaio

A Verifiable Marketplace for Video Intelligence

VERSION 1.0 · SEPTEMBER 2026 · BITTENSOR SUBNET 85

Contents

Abstract

1. Introduction

- 1.1 The Problem
- 1.2 Three Hard Problems
- 1.3 This Paper

2. System Design

- 2.1 Roles
- 2.2 Epochs
- 2.3 Supported Work
- 2.4 Tracks as Markets

3. Measuring Video Quality

- 3.1 The Measurement Primitive
- 3.2 Perceptual Integrity Checks
- 3.3 CPU Determinism
- 3.4 Scorer Identity

4. Continuous Scoring

- 4.1 Challenge Production
- 4.2 Dispatch and Scoring
- 4.3 The Earning State
- 4.4 Failing Closed

5. Competitions

- 5.1 Enrollment
- 5.2 Sealed Evaluation
- 5.3 Deterministic Ranking
- 5.4 Reward Windows
- 5.5 The Limits of Human Review

6. Market Equilibrium

- 6.1 Strategy Space
- 6.2 Competitive Dynamics
- 6.3 The Reference Miner
- 6.4 Collusion Resistance

7. Correctness Enforcement

- 7.1 The Correctness Problem
- 7.2 Layer 1: The Anchored Epoch Log
- 7.3 Layer 2: Independent Audit
- 7.4 The Two Layers Are Complementary
- 7.5 Chain Integrity and Declared Gaps
- 7.6 Identity: Hotkey Authentication
- 7.7 Public Verifiability

8. Epoch Lifecycle

- 8.1 Timing
- 8.2 Processing Pipeline
- 8.3 Data Publication
- 8.4 Determinism

9. Payout Mechanism

- 9.1 Weights Follow Measured Quality
- 9.2 Emission Pools
- 9.3 Inference Ranking
- 9.4 Competition Payouts
- 9.5 The Burn Sink

10. The Product Loop

- 10.1 Customer Workloads
- 10.2 The Two-Sided Flywheel

11. Mechanism Resilience

- 11.1 Score Gaming and Overfitting
- 11.2 Scorer Corruption
- 11.3 History Rewriting
- 11.4 Silent Downtime
- 11.5 Identity Spoofing
- 11.6 Authority-Miner Collusion
- 11.7 Sybil Registration

12. Alphanomic Properties

13. Conclusion

14. Parameter Reference

References

Abstract

Vidaio is a Bittensor subnet (netuid 85) that operates a decentralized marketplace for video intelligence. It launches with the two operations every video workload needs — compression and upscaling — as its first scored tracks, on machinery built to carry many more: any video task whose output can be measured can become a track. Miners run encoders and restoration models and serve real video work. A central scoring authority measures every result with objective, CPU-recomputable metrics — VMAF and perceptual checks computed with pinned models under a pinned scorer identity — and a commercial front end, vidaio.io, routes paying customer demand through the same measured machinery. Emissions follow measured quality: not reputation, not stake, not a reviewer's opinion.

Correctness is enforced without asking anyone to trust the scorer. Every scored round produces a signed score packet. Each epoch, the authority publishes an epoch log — the weight vector, every input behind it, and a manifest of the audit files that justify it — and anchors the log's digest on-chain. Validators mirror the bytes, verify that their hash equals the anchored digest, and only then submit weights: what validators follow is a digest on the chain, not an API response. Independent auditors re-compute the scores from the published evidence — keyless, on commodity CPU — and publish signed clean-or-disputed verdicts on a public honesty surface.

The design rests on three primitives uncommon in decentralized AI networks: (1) every score is recomputable on CPU from published inputs, so measurement is a fact anyone can check rather than a claim anyone must trust; (2) epoch logs chain digest-to-digest back to genesis, with outages declared as explicit, auditable gap records — downtime is an on-chain fact, never a silent hole; and (3) periodic earning competitions evaluated on sealed, single-use media, with bounded reward windows and deterministic ranking. Emission that cannot be honestly allocated is burned to a canonical sink rather than redistributed.

1. Introduction

1.1 The Problem

Video is the heaviest workload on the internet, and the frontier of what machines can do with it — compress it, restore it, enhance it, generate it — is moving faster than any single vendor can track. Yet the market for video intelligence remains dominated by one-size-fits-all encoders and closed, per-seat AI tools. Quality is subjective marketing: vendors publish their own benchmarks, on their own content, with their own metrics. Costs are opaque. Progress is locked inside a handful of vendors, and a better model built anywhere else has no market to sell into.

A large class of video tasks shares a property that makes them unusually well suited to an open market: **the output can be measured**. Full-reference quality metrics such as VMAF turn "does this look good?" into a number that anyone can recompute from the same inputs. Compression adds a second measurable axis — output size — and restoration a third — how faithfully degraded footage is recovered. Where quality is a number, competition can be a protocol. Vidaio launches with the two operations every video workload needs — compression and upscaling — and treats them as the first entries in an open-ended roster of measured tracks.

1.2 Three Hard Problems

A decentralized video-processing network must solve three problems simultaneously to be viable:

Measurement. "Looks good" must become a number that anyone can recompute. If the metric requires a GPU, a private model, or a privileged key, then the score is a claim rather than a fact, and the network's economics rest on trust in whoever computed it. The metric pipeline must be objective, deterministic, CPU-computable, and pinned — and the identity of the code that computed each score must itself be a cryptographic fact.

Trustless payout. Emissions must follow measured quality without trusting the measurer. Any network with a central scorer faces the same objection: what stops the scorer from publishing whatever numbers it likes? The answer cannot be "reputation." It must be structural — published evidence, on-chain commitments, and independent recomputation that makes a dishonest score a provable, permanent, public fact.

Real demand. Synthetic benchmarks get overfit. A network that optimizes only its own test set produces miners that are excellent at the test set. The incentive gradient must point at real customer workloads: fresh content that cannot be memorized, and a commercial pipeline that routes paying demand through the same machinery that scores the miners.

1.3 This Paper

This paper describes Vidaio's mechanism design, presented through its two launch tracks: how video quality is measured so that anyone can recompute the measurement (§3), how miners earn through continuous scoring (§4) and periodic competitions (§5), how the market equilibrates (§6), how correctness is enforced through anchored epoch logs and independent audit (§7), how each epoch proceeds (§8), how emissions are allocated and bounded (§9), and how the commercial product loop closes the incentive circuit (§10). The system operates as Bittensor Subnet 85, inheriting Bittensor's consensus, staking, and emission infrastructure while introducing measurement and audit primitives specific to video intelligence. Every mechanism described here is implemented in the public repositories (miner-vidaio, validator-vidaio) unless explicitly marked **ROADMAP**.

2. System Design

2.1 Roles

Miners (suppliers). Miners run encoders and restoration models, register a hotkey on the subnet, declare the track they compete in, and serve video work dispatched to them. A miner's warrant names one track — compression or upscaling — and every score it earns is bucketed there. Miners are independent operators who bear their own infrastructure costs; an ffmpeg-capable machine and a registered hotkey are enough to start.

The scoring authority (protocol-operated). A central, protocol-operated service dispatches challenges, measures miner outputs with the real metric pipeline, folds scores into each miner's earning state, runs competitions, and at each epoch close publishes one immutable epoch log containing the weight vector and every input behind it. The authority is operated by the protocol team, not by miners or validators. This is a deliberate design choice, and it is legitimate for a specific reason: everything the authority publishes is recomputable, content-addressed, and anchored on-chain, so the authority's honesty is checked by machinery, not assumed (§7). Centralized measurement does not mean discretionary scores.

Thin validators. Validators are the on-chain identities that submit weights — and deliberately nothing more. A validator fetches the epoch-log pointer, mirrors the log bytes from public object storage, verifies that the bytes hash to the digest anchored on-chain, and submits the authenticated weight vector under its own hotkey. Validators never score, never sample miners, and never improvise: a validator that cannot verify holds rather than submits. Because every validator verifies the same anchored bytes and runs the same deterministic math, all validators submit byte-identical vectors, and consensus lands exactly on the published allocation.

Independent auditors. Separately from weight submission, auditor processes re-compute the published scores from the published evidence — keyless, on CPU, with no privileged access of any kind — re-fold each miner's earning state from the

audited packets, re-derive the full weight vector, and publish hotkey-signed verdicts (clean, disputed, or inconclusive) to a public audit surface. Two independent audit modes run per validator identity (§7.3).

The challenge service (protocol-operated). Ingests licensed source content, screens it against a public-corpus fingerprint index, produces degraded challenge payloads, and holds the pristine reference in a sealed store that miners can never reach. Every challenge clip is used once.

The commercial gateway. The customer-facing ingress behind vidaio.io: real compression and upscaling jobs, submitted by paying customers, routed to the miner fleet through the same measured machinery that the scoring pipeline uses (§10).

2.2 Epochs

All scoring, payouts, and weight submission operate on an epoch cycle. An epoch is **360 Bittensor blocks (approximately 72 minutes at the protocol's 12-second block time)**. During each epoch, the authority runs scoring rounds continuously; at the epoch's chain-proven close block it freezes the folded state, builds the epoch log, publishes it to object storage, and anchors its digest on-chain; validators then verify and submit, and auditors advance over the newly published evidence (§8).

2.3 Supported Work

The network launches with two scored tracks:

TRACK	TASK	OUTPUT CODECS	OBJECTIVE
Compression	Re-encode a video payload	AV1, HEVC, H.264, VP9	Smallest output that holds the quality floor (VMAF ≥ 90 against the reference; output must shrink the input by at least 1.25 $\times$)
Upscaling	Restore degraded, low-resolution footage at 2 $\times$ or 4 $\times$	AV1, HEVC, H.264, VP9	Highest restoration fidelity (VMAF, gated at 50, with perceptual checks), within per-factor output size caps

Both tracks are scored by the same pipeline discipline: the full formulas are given in §3 and the exact thresholds in §14.

These two are the starting roster, not the ceiling. A track, in Vidaio's design, is any video task with three properties: a recomputable quality measurement, a challenge supply that cannot be memorized, and a reference implementation to beat. Everything else in this paper — the earning fold, the epoch log, the audit machinery, the competition lineage — is written against tracks in the abstract, not against these two tasks in particular. Further tracks across the video-intelligence frontier, including generative video features, are planned **ROADMAP**; each will arrive the same way the first two did, with its metrics pinned, its scores recomputable, and its own market alongside the existing ones.

2.4 Tracks as Markets

Each track operates as a distinct market. The inference emission pool is split at a fixed ratio between the tracks (0.8 compression / 0.2 upscaling), and an empty or below-floor track never donates its share to the other — unallocated share goes to the burn sink (§9.5), so one track's weakness cannot inflate the other's rewards. Each track also carries its own competition lineage: its own version-zero baseline, its own sequence of competitions, and its own serving champion. A breakthrough in compression neither dilutes nor subsidizes upscaling. This mirrors the economic reality that different tasks attract different specialists with different hardware and different research frontiers — and it is what makes the track roster extensible: a future track joins as a new market with its own emission share, its own baseline, and its own competition lineage, without disturbing the markets already running.

3. Measuring Video Quality

3.1 The Measurement Primitive

Measurement is per-track: each track pins the metric pipeline appropriate to its task, under the same discipline — objective, deterministic, CPU-recomputable, identity-pinned. For the launch tracks, every scored item rests on **VMAF** (Video Multi-method Assessment Fusion), the open-source full-reference perceptual quality metric developed by Netflix and maintained as libvmaf. VMAF compares a candidate video against a reference frame by frame and fuses several elementary quality measures into a 0–100 score that tracks human perceptual judgment. It is deterministic, open, and computable on commodity CPU — the three properties the measurement problem of §1.2 demands.

The scorer runs *two* pinned VMAF models on every item: the primary model and its "no enhancement gain" (NEG) variant, which clips the score inflation that sharpening and contrast tricks can buy. If the two models disagree by more than 3.0 points, the item is zeroed — a large primary-versus-NEG delta is the signature of a miner gaming the metric rather than improving the picture. Critically, this delta is measured against **the payload the miner actually received**, not the pristine original: degradations that the challenge itself introduced (a tone shift, an exposure push) are not held against a miner who faithfully processed what it was handed. This basis was adopted after a controlled calibration study across synthetic and real camera footage; the shipped scorer records the basis and a canonical digest of its input plan in every score packet.

The per-track score compositions:

```
compression:  score = min(1, (0.7·compression_gain + 0.3·vmaf/100) / 1.12)
               zero if output_bytes/input_bytes ≥ 0.80  (must shrink ≥ 1.25×)
               zero if vmaf < threshold - 5    (threshold 90, hard floor 85)

upsampling:   score = 0.1 · exp(6.979 · (0.5·s_quality + 0.5·s_length - 0.5))
               s_length = log(1 + seconds) / log(321), clamped to 1
```

```
zero if vmaf/100 < 0.5  
zero if output exceeds size cap (8× input for 2×, 20× for 4×)
```

These formulas, their constants, and every gate threshold ship in the public repositories and appear in the parameter reference (§14).

3.2 Perceptual Integrity Checks

VMAF alone is not sufficient: a metric that fuses luma-domain features can be blind to manipulations a human notices instantly. The scorer therefore runs a battery of CPU perceptual checks on sampled frames alongside VMAF, and these checks are *required* — a pipeline without them refuses to score rather than scoring partially. They include tone-distribution checks (mean and standard-deviation drift bounds against the reference), grayscale and chroma-integrity checks (a miner cannot desaturate or drop color the reference had), chroma-energy ratio bounds, and a perceptual similarity measure (PieAPP) computed on CPU over a sampled frame window. An item that fails a check is zeroed with a typed gate violation recorded in its score packet — a structured fact an auditor can recompute, never a silent judgment.

3.3 CPU Determinism

Every metric in the pipeline is computed on CPU with pinned model files and pinned tool versions. This is a deliberate constraint with one purpose: **anyone can recompute any score**. No GPU, no proprietary weights, no privileged key. An auditor holding the published challenge input, the miner's output, and the score packet re-runs the same pinned pipeline on commodity hardware and must arrive at the same numbers. Determinism is what converts the audit story of §7 from a spot-check into a proof: a recomputed score either matches or it does not, and a mismatch is conclusive.

3.4 Scorer Identity

The scoring worker mints an effective identity string of the form `vidaio-scorer/1+<digest12>`, where the digest binds the runtime commitment of the scoring code and configuration — every lever that could change a score. This identity is stamped into every score packet, published on the worker's health endpoint, and bound into every challenge commitment before dispatch. It moves whenever any scoring lever moves: "which scorer measured this" is a cryptographic fact, not a label.

The system fails closed on identity mismatch. A score request that names a different scorer is refused rather than answered by the wrong one. The challenge service continuously re-verifies the live worker's identity and disables challenge production if it ever changes unexpectedly. A validator pins the scorer identity it first discovers and refuses to fold packets from a different scorer into the same earning state without explicit operator acknowledgment. There is no path by which scores from an unstated pipeline version can silently enter the economic record.

4. Continuous Scoring

4.1 Challenge Production

Fresh challenge content is continuously ingested from licensed sources. Each candidate asset passes through an eligibility pipeline before it can become a challenge:

- **Fingerprint screening.** A CPU-computed 64-bit perceptual video hash is checked against a durable index of public benchmark and training corpora. Content perceptually close to anything a miner could have trained on or memorized is rejected. Production requires the index to be present and integrity-pinned by its exact SHA-256.
- **Degradation.** For each challenge, the service derives the miner-facing payload (for upscaling, a degraded low-resolution derivation of the pristine source; for compression, the encode payload) through a deterministic media pipeline whose plan is committed.
- **Sealed reference.** The pristine reference lives in a validator-only directory tree that is structurally disjoint from the miner-visible tree — no reference artifact ever shares a parent directory with anything a miner can reach.
- **Single use.** Every challenge clip is retired after one use. There is nothing to overfit: the next round is always fresh content.
- **Commitment before dispatch.** Each challenge's commitment — binding the input, the reference, the scorer identity, and the challenge parameters — is anchored on-chain and its finalized receipt is signed into the miner exchange, so the challenge that was served is provably the challenge that was scored.

4.2 Dispatch and Scoring

The authority's round loop dispatches challenges to registered miners on their advertised endpoints. The exchange is mutually authenticated: the dispatching validator identity signs the request and the input facts; the miner signs its output; both sides verify the other's hotkey against the live chain registry (§7.6). Outputs are transferred with length and digest verification, then handed to the scoring worker, which produces an audit-grade **score packet**: the metric values, gate outcomes, the scorer identity, and digests binding the exact input and output bytes. Packets are committed to a content-addressed audit store and their digests are folded into a Merkle publication root — the leaves the epoch log later commits to (§7.2).

A miner that fails to respond does not simply vanish from the record: signed, miner-attributable availability failures contribute real zero observations to the earning state, so selective non-response cannot freeze a favorable standing. Failures attributable to the authority's own infrastructure are excused and never scored against the miner.

4.3 The Earning State

Scores fold into a per-miner exponential moving average — the earning state that ranks miners within their track:

$$\text{ewma_new} = 0.75 \cdot \text{ewma_old} + 0.25 \cdot \text{round_score}$$

The decay constant (0.75) balances responsiveness against stability: sustained strong performance moves a miner up within a handful of rounds, while a single lucky round cannot vault anyone to the top. A miner must hold an absolute earning state of at least 0.10 to receive any payout, and only the top five miners per track earn in a given epoch (§9.3) — the market rewards being among the best, not merely being present. The fold is stateful and history-dependent, which is precisely why it is computed once, centrally, and published with every input needed to re-fold it (§7.3): two parties folding the same packet sequence must arrive at the same state, and auditors verify that they do.

4.4 Failing Closed

Every fault path in the scoring pipeline resolves to refusal, never to a substituted value. A scorer-identity mismatch refuses production. An oversized input is refused before a byte is written. A metric that cannot run produces a typed violation, not a partial score. An epoch whose authoritative inputs are unavailable or ambiguous is held, not improvised (§8.4). The system's uniform answer to "something is wrong" is to stop and say so loudly — because in an economic protocol, a quiet wrong number is strictly worse than a loud absence.

5. Competitions

Continuous scoring rewards operating well. Competitions reward *moving the frontier*. Periodically, per track, the subnet runs an earning competition: contenders submit their code, the code is built and evaluated on sealed single-use media against the incumbent baseline, and the winner earns a bounded reward window. Competitions are how a genuinely better encoder is worth money the day it exists.

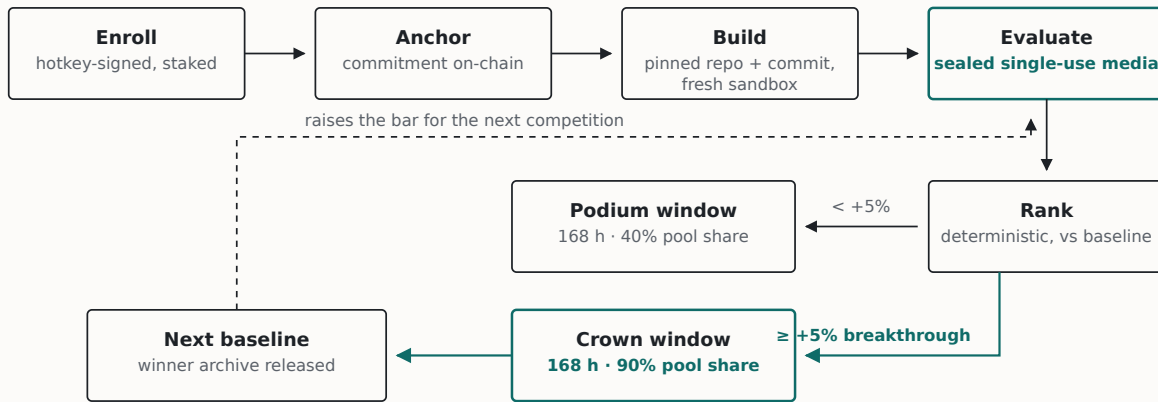


Figure 1 — the competition lifecycle. A contender enrolls by signing with its own registered hotkey, its exact source identity is committed on-chain before evaluation, and it is judged on sealed media used exactly once. A winner that beats the incumbent baseline by the breakthrough margin earns the crown window and — only after its archive is publicly released — becomes the baseline every future contender must beat.

5.1 Enrollment

Enrollment is permissionless and self-authenticating. A contender enrolls by signing its own enrollment payload with a hotkey that is registered on the subnet and holds at least the minimum enrollment stake (a protocol parameter set at launch sign-off; §14). No operator vouches for a contender, and no operator can forge one: the enrollment is refused unless the signature verifies against the chain's own registry.

A contender submission is a precise, immutable identity: a repository URL, an exact commit and tree hash, and the digest of the execution image its code will run in. These are committed before evaluation begins. What competed is never a matter of recollection.

5.2 Sealed Evaluation

Contender code is built in a fresh, isolated sandbox from the pinned source identity, and evaluated on a hidden set of single-use media prepared specifically for the competition — never drawn from the continuous challenge corpus, never reused, and unknown to every contender (and to the baseline) until evaluation runs. The incumbent baseline executes over exactly the same hidden item matrix as every contender, under the same pipeline, so the comparison is symmetric by construction. After results are finalized, the pristine references are published so any third party can re-run the evaluation on CPU and confirm every number (§7.3).

5.3 Deterministic Ranking

A contender's economic score is the arithmetic mean of its complete per-item packet scores — the same CPU-recomputable packets the continuous pipeline produces. The per-item aggregate weighs media quality (0.6), cost efficiency (0.25), and length coverage (0.15), with worst-decile aggregation so a contender cannot hide catastrophic items behind a good average: the worst tenth of items is what the aggregation examines. Contenders sort by score with hotkey and uid as deterministic tie-breaks. Duplicate suppression is derived mechanically from coldkey, advertised IP, and exact output matrix — a contender cannot enter twice behind two identities and take two prizes. No human ranking field exists anywhere in this computation (§5.5).

5.4 Reward Windows

A completed competition opens a bounded reward window of **168 hours (7 days)**, protocol-pinned, half-open from the epoch-close timestamp at which the result was applied. The window's character depends on the margin over the incumbent baseline:

- **Podium window** — the winner improved on the baseline by less than the 5% breakthrough floor. The competition pool takes 40% of miner emissions for the window; the top three contenders split it 0.70 / 0.20 / 0.10.
- **Crown window** — the winner met or exceeded the 5% floor: a genuine breakthrough. The competition pool takes 90% of miner emissions for the window, concentrated on the result that moved the frontier. A crown result earns only after the winner's exact submission archive has been publicly released — the network pays for breakthroughs it can inspect, and the released winner becomes the next baseline.

When the window closes, emissions return to the idle allocation (§9.2). Windows do not stack and do not extend; a new result replaces the old window. The bar permanently rises — the reward is finite.

5.5 The Limits of Human Review

Competitions include a human review period before completion. Its scope is deliberately and structurally narrow: **operational inspection only — it cannot move money**. Review fields (manual ranks, disqualification flags, eligibility notes) are not inputs to the economic computation; the payout derivation reads only packet scores, and auditors recompute it from packet scores, so a review edit that attempted to redirect emissions would produce a provable derivation mismatch (§7.3). The only human lever over competition economics is a global, visible emergency stop for competition emissions — an on/off switch whose state is public, not a dial.

6. Market Equilibrium

6.1 Strategy Space

A miner's strategies are exactly the ones the protocol wants to exist. On the continuous market: better encoder parameters, better rate-control, better restoration models, faster and cheaper serving — anything that raises measured output quality on fresh content. In competitions: the same, plus cost efficiency and length coverage, which enter the aggregate directly. There is

no routing bid to game, no benchmark set to memorize, and no reputation system to court. The single input to a miner's income is the measured quality of its output on content it has never seen.

6.2 Competitive Dynamics

The graded top-five rank curve (§9.3) makes the continuous market a tournament rather than a participation reward: only the best five miners per track earn, on a 5:4:3:2:1 curve. The distance between rank one and rank six is the distance between earning and not earning, which keeps competitive pressure on the top of the field rather than the bottom. Because the earning state is an EWMA over fresh single-use content, standing decays without sustained performance — an incumbent cannot coast, and a superior entrant displaces an incumbent within days, not months.

Competitions compound this: any contender who can beat the published, runnable baseline has a permissionless path to the crown window. The equilibrium behavior for a rational, capable actor is exactly the behavior the network exists to elicit — ship the better encoder, in the open, and be paid for the improvement.

6.3 The Reference Miner

The protocol publishes a complete, honest reference miner in the open repository (miner-vidaio): an ffmpeg-based implementation of both tracks that serves real work end-to-end. It plays the market-maker role. In the network's early life it guarantees the pipeline has supply; permanently, it is the floor every entrant must beat honestly — a public, runnable statement of "this much quality is table stakes." The version-zero competition baselines are seeded from it. Anyone who cannot outperform the published reference has no economic claim on the network; anyone who can has a public yardstick proving it.

6.4 Collusion Resistance

Overfitting fails structurally. The continuous corpus is single-use and fingerprint-screened against public datasets; competition media is sealed, fresh, and used once. A miner cannot train on the test set, because the test set does not exist until the moment it is used and is retired the moment after.

Copying a winner is allowed — and healthy. A crown winner's archive is public before it earns. Anyone may run it, study it, and build on it. This is deliberate: copying moves the whole field onto the new frontier, at which point the copy earns nothing exceptional (the baseline has risen to meet it) and only a further improvement pays. The network converts each breakthrough into public infrastructure and re-aims the incentive at the next one — the open-source ratchet, priced.

Colluding with the scorer is the remaining theory, and §7 is its answer: every score is recomputable by outsiders from published evidence, so a corrupted score is not a secret between two parties — it is a permanent, provable, public artifact.

7. Correctness Enforcement

7.1 The Correctness Problem

In Vidaio, the number that moves money is a quality score, and the entity that computes it is a central authority. The correctness problem is therefore blunt: *what stops the authority from publishing whatever scores it likes?* A decentralized network cannot answer "its reputation." Vidaio answers with two enforcement layers that make a dishonest score either impossible to act on (layer 1) or impossible to hide (layer 2) — and with a history structure that makes the record itself tamper-evident all the way back to genesis.

7.2 Layer 1: The Anchored Epoch Log

At every epoch close, the authority publishes one immutable artifact: the **epoch log**. It contains the complete weight vector (as exact u16 chain weights), the full miner census at the close block, every miner's earning state *and the per-packet inputs it folds from*, any applied competition result with its complete evidence matrix, and an **audit manifest** naming exactly which content-addressed audit files back each weight. The log is canonical JSON; its SHA-256 digest is its identity.

That digest is **anchored on-chain** through Bittensor's commitments mechanism before validators act on it. The validator path is then a chain of verifications, not a chain of trust:

```
sha256(mirrored bytes) == published pointer digest == on-chain anchored digest
```

A validator fetches the epoch pointer from the authority's API, mirrors the log bytes directly from public object storage, and verifies the three-way digest equality — the on-chain leg is mandatory. Only then does it submit the log's weight vector under its own hotkey. If the epoch is finalized but not yet anchored, the validator holds. If any leg of the equality fails, the validator refuses to submit and raises an alarm: a digest mismatch is evidence of tampering, not an operational hiccup. **What validators follow is a digest on the chain, not an API response.** The authority cannot serve one log to the chain and another to the fleet; it cannot revise a log after anchoring it; and it cannot make validators act on bytes it is unwilling to commit to publicly and permanently.

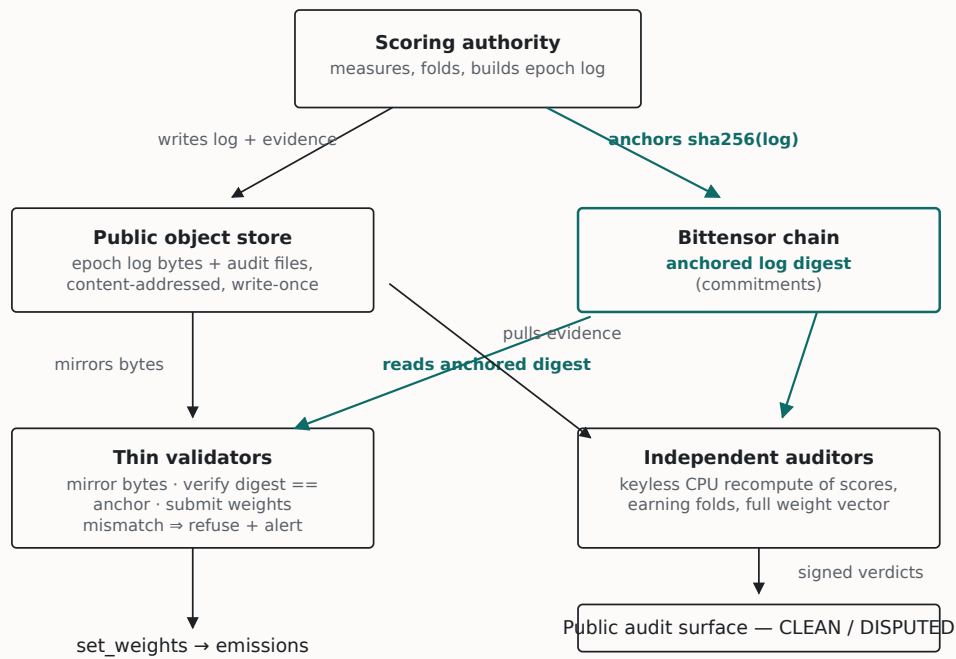


Figure 2 — the two enforcement layers. The authority can only move emissions through bytes it has published and a digest it has anchored on-chain (layer 1); every economic claim in those bytes is then independently recomputed from the published evidence by keyless CPU auditors whose signed verdicts are public (layer 2).

7.3 Layer 2: Independent Audit

Anchoring proves the log was not altered after publication. Layer 2 asks the harder question: *was the log honest when it was written?* Independent auditors answer it by recomputing everything, from the outside, with no privileged access:

- **Every score, re-measured.** For each committed item, the auditor fetches the exact challenge input, miner output, and score packet named by the audit manifest, and re-runs the real scoring engine on CPU. A tampered score fails with a typed mismatch; a packet outside the committed Merkle set fails inclusion; edited metadata, a broken commit-reveal, or a substituted track each fail with their own conclusive code.
- **Every earning state, re-folded.** Honest packets with an inflated EWMA would survive a per-packet check — so the auditor re-folds each paid miner's earning state from the audited packet scores plus the carry-in chained from prior epochs, all the way from genesis. The authority cannot substitute earning state without the fold disproving it.
- **The whole vector, re-derived.** The auditor independently rebuilds the weight vector — census, deduplication, track assignment, rank curve, competition result, reward-window state, quantization — from verified inputs, and compares digests with the published vector. Competition evidence gets the same treatment: every subject's packets recomputed, the ranking re-derived, the baseline's provenance verified.
- **Signed, public verdicts.** Each auditor posts a hotkey-signed report — CLEAN, DISPUTED, or INCONCLUSIVE, with per-item verdicts and typed reason codes — to the audit results service, whose read side is fully public. The aggregate

verdict is DISPUTED if *any* auditor proved a fault; a dispute cannot be out-voted, and an all-skip audit is INCONCLUSIVE, never washed to clean. A disputed epoch is an alarm the whole network sees.

Two audit modes run per validator identity, in processes isolated from weight submission. The *own-audit* mode recomputes every committed item — full coverage, so its conclusion is not probabilistic. The *beacon* mode selects its work as a pure function of a post-finalization future block hash the authority could not have known when it built the manifest, so the authority cannot steer any third-party sample away from a poisoned item. Because the entire audit runs keyless on CPU from public data, anyone — not only registered validators — can perform it.

7.4 The Two Layers Are Complementary

PROPERTY	LAYER 1: ANCHORED LOG	LAYER 2: INDEPENDENT AUDIT
Question answered	Is this the exact artifact that was published?	Was the published artifact honest?
Timing	Before any validator acts	Continuously, after publication
Mechanism	Three-way digest equality against the on-chain anchor	Keyless CPU recomputation from published evidence
Coverage	The full log, byte-exact	Every committed item, every earning fold, the full vector
On failure	Validators refuse to submit; the last confirmed vector stands	Signed DISPUTED verdict on the public surface; permanent cryptographic trail
Who can run it	Every validator, every epoch	Anyone with a CPU

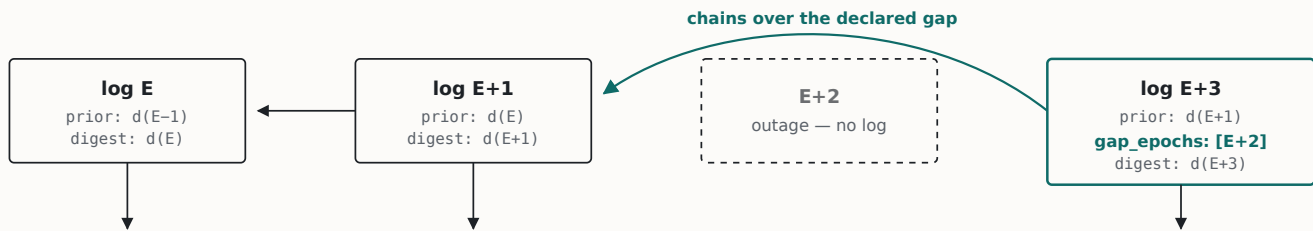
Layer 1 makes a forged artifact unusable: nothing unanchored moves weights. Layer 2 makes a dishonest artifact undeniable: the evidence to convict it is published by the authority itself, as a precondition of moving weights at all. Together they close the loop that pure post-hoc audit systems leave open — the authority cannot choose between "act on it" and "hide it," because acting on it *is* publishing it.

7.5 Chain Integrity and Declared Gaps

Each epoch log carries the digest of its predecessor. The logs form a hash chain from the current epoch back to genesis, with each link independently anchored on-chain: rewriting any historical epoch would break every subsequent digest and contradict the anchors — a rewrite is not merely detectable but self-announcing.

Outages get the same discipline. If the authority is down when an epoch's anchor window elapses, the next published log **explicitly declares the missed epochs** in a gap record: contiguous, anchor-verified, and auditable, preserving the digest chain

across the hole. Downtime is thereby an on-chain fact with a first-class record — never a silent gap a reader must notice by its absence, and never an opportunity to quietly rebuild history. Auditors verify gap declarations like any other claim: a declared gap must correspond to genuinely absent anchors, and an undeclared hole breaks the chain loudly.



Bittensor chain — one anchored digest per published log; no anchor exists for E+2, matching its declaration

Figure 3 — the digest chain with a declared gap. Every log names its predecessor's digest, and every digest is anchored on-chain. An outage produces an explicit, auditable `gap_epochs` declaration in the next log — the chain remains intact and history cannot be quietly rewritten or quietly thinned.

7.6 Identity: Hotkey Authentication

Every miner- and validator-facing surface in Vidaio authenticates callers by their **registered hotkey** — identity is the chain's, not a password's. Two schemes cover the two traffic shapes:

- **Scheme A — signed requests**, for low-rate, high-value calls (challenge resolution, competition enrollment): the caller sends its hotkey, a timestamp, a nonce, and a signature over a domain-separated canonical digest of the method, path, body hash, timestamp, and nonce. The server verifies the signature, checks the hotkey against a cached view of the live subnet registry, and rejects replays within a bounded window.
- **Scheme B — challenge-minted session tokens**, for high-rate polling (epoch pointers): the caller solves a server-issued challenge with a Scheme A signature and receives a short-lived, server-signed session token binding its hotkey and scope. Registration is re-checked from the cached registry on every request, so **deregistration revokes access within the registry's refresh interval** — leaving the subnet means leaving its APIs.

The miner exchange itself is mutually signed end-to-end: the dispatching validator signs the request and input facts, the miner signs its output, and each side verifies the other against the chain. Enforcement is fail-closed throughout — an unreachable registry refuses service rather than waving callers through — and mainnet preflight requires enforcement on. Deliberately public surfaces (the dashboard, audit reads, released evidence, miner health probes) remain keyless, because the audit story of §7.3 requires that anyone can check the network without asking permission.

7.7 Public Verifiability

The sum of the above is a network whose every economic fact is checkable from public data: the epoch logs and their evidence in public object storage, the anchors on the public chain, the audit verdicts on a public read surface, the scoring code and its constants in public repositories, and the pristine competition references released after results. A journalist, a competitor, or a token holder — none of whom hold any key — can independently confirm that every unit of emission in any epoch followed a measured, recomputable score. No comparable claim is available from any closed video vendor, whose quality numbers are self-reported and whose evaluation sets are private.

8. Epoch Lifecycle

8.1 Timing

Each epoch spans 360 Bittensor blocks — approximately 72 minutes at the 12-second block time. Epoch boundaries are taken from consensus-finalized chain state, never projected from a local clock or an assumed grid, so every participant resolves the identical close block for every epoch.

8.2 Processing Pipeline

1. **Measure (continuous).** The authority dispatches single-use challenges, collects signed outputs, scores them into committed packets, and folds each miner's earning state.
2. **Close.** At the chain-proven close block, the state is frozen: the miner census, the earning states through exactly that block, and any newly applied competition result.
3. **Finalize.** The finalizer composes the weight vector, quantizes it to the exact u16 chain grid, assembles the epoch log with its complete audit manifest, and writes it to public object storage — content-addressed, with a finalization marker written last so a half-written log can never be read as complete.
4. **Anchor.** The log's digest is committed on-chain within the epoch's anchor window.
5. **Verify and submit.** Each validator mirrors the bytes, verifies the three-way digest equality (§7.2), and submits the authenticated vector under its own hotkey, gated by the chain's rate limits.
6. **Audit.** Auditor processes advance over the new log on their own cadence and post signed verdicts (§7.3).

8.3 Data Publication

All published artifacts are content-addressed — the digest is the identity — and written once: the object store's writers use conditional creates, so an existing object cannot be silently replaced, and reads verify content against the addressed digest before returning it. The epoch log, the score packets, the audit bundles, and the competition evidence all live in this store, and the epoch log's manifest is the index that tells any outsider exactly which objects justify which weight.

8.4 Determinism

Given the same inputs, the pipeline produces byte-identical outputs. The weight computation has no randomness, no wall-clock dependence, and no platform dependence: every ordering carries an explicit uid tie-break, money-deciding arithmetic runs in exact rational form rather than floating point, and the float-to-u16 quantization is a single shared implementation used by the authority, the validators, and the auditors alike. The quantized vector sums to exactly 65,535 with every positive earner floored to at least one unit. Two parties who hold the same epoch log will produce — not approximately, but byte-for-byte — the same on-chain weight vector, which is what makes a divergent submission provably wrong rather than arguably different.

When an authoritative input is unavailable or ambiguous — the pointer unreachable, the anchor missing, a digest in disagreement — a validator holds: it submits nothing, and the last confirmed vector persists on-chain. No participant ever improvises a locally computed vector, because a plausible wrong answer is the only thing worse than no answer.

9. Payout Mechanism

9.1 Weights Follow Measured Quality

The on-chain weight vector — the object that directs miner emissions — is a pure, deterministic function of measured quality: the per-track earning states from continuous scoring, plus any active competition reward window. No stake enters it (buyable stake must not become weight), no reputation enters it, and no discretionary field exists anywhere in its derivation. The vector is published in the epoch log with every input needed to recompute it, and auditors do (§7.3).

9.2 Emission Pools

Miner emissions divide between an inference pool (continuous scoring) and a competition pool (reward windows), in fixed proportions per state:

STATE	INFERENCE POOL	COMPETITION POOL	CANONICAL SINK
Idle — no active window	80%	0%	20%
Podium window — winner below the breakthrough floor	60%	40%	unpayable shares only
Crown window — breakthrough ($\geq$ 5% over baseline)	10%	90%	unpayable shares only

Each state's allocation sums exactly to one, and the proportions are protocol-pinned. The idle state's 20% sink share is structural — in the network's resting state, one-fifth of miner emission is burned rather than distributed, a standing statement that emission unmatched by exceptional work is removed, not smeared across participants.

9.3 Inference Ranking

The inference pool splits 0.8 / 0.2 between the compression and upscaling tracks in every state. Within each track, miners rank by earning state (uid as the deterministic tie-break), subject to three eligibility screens: an absolute earning-state floor of 0.10, exclusion latching for misbehavior until a genuine score re-enters a miner at zero, and deterministic deduplication by coldkey and advertised IP so one operator cannot occupy multiple paid ranks. The top five eligible miners per track then split the track's share on a graded curve:

rank 1 : 2 : 3 : 4 : 5 → 5 : 4 : 3 : 2 : 1 (rank ≥ 6 earns zero)

An empty or below-floor track does not donate its share to the other track; it goes to the sink.

9.4 Competition Payouts

During an active reward window, the competition pool pays the top three contenders of the window's result 0.70 / 0.20 / 0.10. Payment binds to the *hotkey*: a podium hotkey still registered at each epoch's close is paid at its current uid, while a hotkey that has left the subnet forfeits its fixed share to the sink — never to the other contenders. A missing rank is likewise sunk, not redistributed. When the window's 168 hours elapse, the state returns to idle.

9.5 The Burn Sink

Every fixed share that cannot be honestly allocated — the idle state's 20%, an empty or below-floor track, an incomplete podium, a departed reward-window hotkey — is routed explicitly to the canonical sink: the subnet-owner UID resolved live from the chain, the recognized burn destination under Bittensor's emission mechanics. The explicit routing matters because the chain normalizes submitted weights: merely omitting an unallocated pool would silently donate it to the remaining earners. Vidaio makes the complete allocation explicit so that normalization cannot cross-subsidize anyone. In the limiting case — an epoch with no eligible earners at all — the published vector is 100% sink: the network would rather burn an epoch's emission than pay it to unmeasured work.

Two locked non-levers complete the picture: there is no discretionary burn percentage (the sink receives only structurally unallocatable shares), and stake weighting in the economic vector is fixed at zero. An operational failure is never converted into a sink allocation — it holds the epoch instead (§8.4). There are no hidden operator levers on the economic vector.

10. The Product Loop

10.1 Customer Workloads

vidaio.io sells video processing to real customers — a dashboard, an API, and credit-based billing — starting with compression and upscaling, and serves those jobs through the same measured machinery the network runs on. Customer jobs are dispatched to the miner fleet through the protocol gateway with the same authenticated exchange, the same size and integrity verification, and the same fleet-quality ordering that the scoring pipeline maintains: the miners at the top of the measured ranking are the miners that serve paying work.

10.2 The Two-Sided Flywheel

The product and the network improve each other in a closed loop. Customer demand shapes what the network optimizes: the workloads people actually pay for — their codecs, their resolutions, their content classes — inform the challenge corpus and the competition design, so the incentive gradient points at real-world usefulness rather than at a static benchmark. Demand is also how the track roster grows: the video-intelligence features customers ask for next are the natural candidates for the network's next scored tracks **ROADMAP**. In return, the network gives the product a permissionless, ever-improving supply side: every breakthrough that wins a crown window becomes the new serving baseline, and the product's unit economics improve the day a better encoder exists anywhere in the world. A closed vendor must staff its way to a better encoder; Vidaio's front end inherits one from an open market that pays for it directly.

Revenue mechanics beyond this operational loop — how product revenue interacts with the token beyond funding the network's operation — are deliberately out of scope for this paper **ROADMAP**; the implemented token mechanics are exactly those of §9 and §12.

11. Mechanism Resilience

The following analyses cover the principal ways a rational adversary might attack the mechanism, and the structural property that defeats each.

11.1 Score Gaming and Overfitting

A miner cannot train on the evaluation set because no evaluation set persists. Continuous challenges are licensed fresh content, screened by perceptual fingerprint against public corpora, and retired after a single use; competition media is sealed until evaluation and never reused. Metric-level gaming — sharpening and contrast tricks that inflate a naive score — is caught by the dual-model VMAF delta gate and the perceptual integrity checks (§3.2), which zero the item with a recomputable, typed violation.

11.2 Scorer Corruption

A corrupted or covertly modified scoring pipeline cannot act quietly. The scorer's effective identity digest moves with every lever that could change a score, is bound into every challenge commitment and every score packet, and is enforced fail-closed at every consumer (§3.4) — so an unstated change halts production rather than contaminating the record. A corrupt score

published under the legitimate identity is caught by layer 2: independent CPU recomputation from published inputs yields a mismatch that a signed, public, disputed verdict makes permanent (§7.3).

11.3 History Rewriting

Epoch logs chain digest-to-digest back to genesis, and every digest is anchored on-chain (§7.5). Rewriting any epoch requires either breaking SHA-256 or contradicting anchors already finalized by the chain's consensus. The write-once object store adds a third barrier: published bytes cannot be replaced in place. History in Vidaio is not a database the operator maintains; it is a chain of commitments the operator cannot take back.

11.4 Silent Downtime

An authority outage cannot be hidden or papered over. Missed epochs must be explicitly declared as gap records in the next published log — contiguous, anchor-verified, and auditable — and validators hold during the outage, leaving the last confirmed vector in force. An undeclared hole breaks the digest chain loudly; a declared gap is a public, permanent record that the outage happened. The operator's uptime record is written by the protocol, not by the operator.

11.5 Identity Spoofing

Every economic surface authenticates registered hotkeys with chain-verified signatures — signed requests with replay protection for high-value calls, challenge-minted short-lived sessions for polling, and mutual signing on the miner exchange (§7.6). There are no passwords to steal and no API keys whose theft grants a miner's identity. Deregistration revokes access within the registry refresh interval, and an unreachable registry fails closed.

11.6 Authority–Miner Collusion

Suppose the authority conspired to favor one miner — inflated scores, a doctored earning state, a rigged competition. Every avenue produces published, recomputable evidence against itself: inflated packet scores fail CPU recomputation; a doctored earning state fails the genesis-chained re-fold; a rigged competition ranking fails re-derivation from the committed packets; and omitting the evidence entirely fails the epoch log's construction-time validation that every paid weight be backed by committed evidence. The conspiracy's own payout mechanism forces it to publish the proof of its dishonesty, permanently, under an anchored digest. Layer 2 exists precisely so that this class of attack converts into a public, signed, cryptographically supported dispute (§7.3).

11.7 Sybil Registration

Registering many hotkeys buys nothing. Emissions follow measured quality through a ranked curve, not headcount: additional identities must independently out-perform the field to earn at all, and deterministic deduplication by coldkey and advertised IP excludes duplicate operators from occupying multiple paid ranks in the same track (§9.3). Competition entry

requires per-hotkey stake above the enrollment floor, and duplicate contenders are suppressed by coldkey, IP, and exact output matrix (§5.3). Sybil registration is a pure cost with no return path.

12. Alphanomic Properties

Vidaio's token mechanics are deliberately few, and each is a direct consequence of the payout design rather than a separate program:

Emissions are bounded to measured work. The weight vector is derived exclusively from recomputable quality measurements and protocol-pinned pool shares (§9). There is no path by which capacity, registration, stake, or narrative attracts emission — only measured output on fresh content does.

Unallocatable emission is burned. Every fixed share without an honest recipient — including one-fifth of miner emission in the network's resting state — routes to the canonical sink rather than being redistributed (§9.5). The network's default answer to "no one earned this" is to remove it.

Competitions concentrate reward on breakthroughs. The crown mechanism moves up to 90% of miner emission, for a bounded 168-hour window, onto a result that verifiably improved the field's frontier by at least 5% — and then the baseline rises and the window closes (§5.4). Emission concentration tracks technical progress, not tenure.

This paper makes no claims about token price, and the mechanism contains nothing that depends on one: no live price feed enters the weight computation, and the epoch log carries no economic field a market move could alter.

13. Conclusion

Vidaio is a verifiable marketplace for video intelligence. Its wager is that wherever video quality can be measured, an open market can out-serve closed vendors — and that a measurement anyone can recompute is a foundation strong enough to hang an economy on. It begins with the two heaviest operations in video, compression and upscaling, and its track machinery is built to carry the frontier beyond them.

The design reduces to three claims defended in this paper. First, **measurement is a public fact**: every score is produced by pinned, CPU-deterministic metrics under a cryptographically bound scorer identity, and can be re-derived by anyone from published inputs (§3, §7.3). Second, **the economic record is tamper-evident end to end**: weights move only through epoch logs whose digests are anchored on-chain, which chain digest-to-digest back to genesis, and in which even downtime is a declared, auditable record (§7.2, §7.5). Third, **the incentive gradient points at real progress**: single-use content makes overfitting structurally impossible, a public reference miner sets the honest floor, breakthrough competitions concentrate bounded reward on verified improvements whose winning code becomes public infrastructure, and a commercial front end keeps the whole gradient aimed at work people actually pay for (§5, §6, §10).

The result is a network in which a better encoder is worth money the day it exists, from anywhere in the world, with no gatekeeper to convince — and in which every unit of emission that paid for it can be traced, recomputed, and proven honest by anyone with a CPU.

14. Parameter Reference

PARAMETER	DESCRIPTION	CURRENT VALUE
Subnet	Bittensor netuid	85
Epoch length	Bittensor blocks per epoch	360 blocks (~72 min)
Block time	Bittensor block interval	12 s (protocol constant)
Epoch log schema	Version fence for the log format and weight pipeline	16
Scored tracks	Independent earning markets (launch roster)	compression, upscaling
Track split	Inference pool share per track	0.8 / 0.2
EWMA decay	Earning-state fold: <code>new = d·old + (1-d)·score</code>	d = 0.75
Payout floor	Minimum absolute earning state for any payout	0.10
Paid ranks per track	Graded curve over the top eligible miners	5 (5:4:3:2:1)
Idle allocation	Inference / competition / sink, no active window	0.80 / 0 / 0.20
Podium allocation	Inference / competition during a non-breakthrough window	0.60 / 0.40
Crown allocation	Inference / competition during a breakthrough window	0.10 / 0.90
Podium split	Competition pool over the top three contenders	0.70 / 0.20 / 0.10
Breakthrough floor	Margin over baseline for a crown (inclusive)	5%
Reward window	Duration of a competition earning window	168 h (protocol-pinned)
Enrollment stake floor	Minimum alpha stake to enroll in a competition	set at launch

PARAMETER	DESCRIPTION	CURRENT VALUE
VMAF threshold (compression)	Quality floor; scores taper in a 5-point band below it	90 (hard floor 85)
VMAF gate (upscaling)	Hard floor on the 0-100 scale	50
VMAF model delta	Max primary-vs-NEG disagreement before the item zeroes	3.0
Compression rate cap	Output/input size ratio at/above which the item zeroes	0.80 ($\geq 1.25\times$ shrink)
Compression weights	Score composition: compression gain / VMAF, norm 1.12	0.7 / 0.3
Upscaling formula	Final score exponent / coefficient / length log base	6.979 / 0.1 / 321
Upscaling size caps	Output bytes vs input, per factor	8 $\times$ at 2 $\times$, 20 $\times$ at 4 $\times$
Codec allowlist	Accepted output codecs, both tracks	AV1, HEVC, H.264, VP9
Aggregate weights	Competition item aggregate: quality / cost efficiency / length coverage	0.6 / 0.25 / 0.15
Worst-decile fraction	Share of worst items examined by aggregation	0.1
Scorer identity	Pinned effective identity of the scoring pipeline	vidaio-scorer/1+<digest12>
Auth replay window	Signed-request timestamp/nonce validity	120 s
Session token lifetime	Scheme B token expiry	3600 s
Registry refresh	Hotkey-registry cache interval (revocation bound)	45 s
Weight grid	Quantized vector sum (exact)	65,535
Burn sink	Destination of unallocatable emission	subnet-owner UID, resolved from chain

Values marked "set at launch" are pending launch-configuration sign-off. Operator-tunable service limits (payload caps, timeouts, cache intervals beyond those listed) are documented in the repositories.

References

Rao, Y. (2021). *Bittensor: A Peer-to-Peer Intelligence Market*. <https://bittensor.com/whitepaper>

Opentensor Foundation. *Bittensor Documentation*. <https://docs.learnbittensor.org>

Li, Z., Aaron, A., Katsavounidis, I., Moorthy, A., & Manohara, M. (2016). *Toward a Practical Perceptual Video Quality Metric*. Netflix Technology Blog.

Netflix. *VMAF — Video Multi-method Assessment Fusion (libvmaf)*. <https://github.com/Netflix/vmaf>

Prashnani, E., Cai, H., Mostofi, Y., & Sen, P. (2018). *PieAPP: Perceptual Image-Error Assessment through Pairwise Preference*. Proceedings of CVPR 2018.

Alliance for Open Media (2018). *AV1 Bitstream & Decoding Process Specification*. <https://aomedia.org>

ITU-T (2021). *Recommendation H.265: High Efficiency Video Coding*.

ITU-T (2021). *Recommendation H.264: Advanced Video Coding for Generic Audiovisual Services*.

Grange, A., de Rivaz, P., & Hunt, J. (2016). *VP9 Bitstream & Decoding Process Specification*. Google.

Vidaio. *miner-vidaio — run a miner; reference implementation included*. <https://github.com/vidAio-subnet/miner-vidaio>

Vidaio. *validator-vidaio — verify everything*. <https://github.com/vidAio-subnet/validator-vidaio>